**Module 3**

**Plotting and Visualization**

Developing clear and insightful visualizations—often referred to as plots—is a key element of data analysis. Visuals serve different purposes: they can be used in the early stages of exploration to detect anomalies or assess the need for data transformation, or they may act as the final product, such as interactive dashboards for web-based applications.

In Python, a wide range of libraries is available to support both static and interactive visualizations. Among these, matplotlib is one of the most widely used libraries, especially for creating high-quality 2D visualizations suitable for publication. It was originally developed by John Hunter in 2002 to replicate MATLAB-style plotting functionality in Python.

Over the years, the matplotlib project has worked closely with the IPython/Jupyter ecosystem to make it easier to create interactive plots directly within Jupyter notebooks. The library is compatible with various graphical user interface (GUI) backends across different operating systems and supports exporting visuals in numerous vector and image formats, such as PDF, SVG, PNG, JPG, GIF, BMP, and others.

Most of the figures presented in this textbook were generated using matplotlib. Furthermore, a number of higher-level visualization libraries have been built on top of matplotlib. One such popular library is seaborn, which is discussed later in the chapter.

To work with matplotlib interactively in a Jupyter notebook, you can activate inline plotting by executing:

```
%matplotlib notebook
```

This enables a dynamic interface for generating and interacting with plots directly within the notebook environment.

## 1. A Brief matplotlib API Primer

### 1.1. Figures and Subplots

To start using matplotlib, a popular Python library for visualizations, you typically import its pyplot module with the following line of code:

import matplotlib.pyplot as plt

If you're working within a Jupyter Notebook, enabling inline or interactive plotting is helpful. You can do this using:

%matplotlib notebook

This sets up the notebook to render plots directly within the interface. Once configured, you can begin creating basic plots such as line graphs. For instance:

```
In [12]: import numpy as np

In [13]: data = np.arange(10)

In [14]: data
Out[14]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

In [15]: plt.plot(data)
```

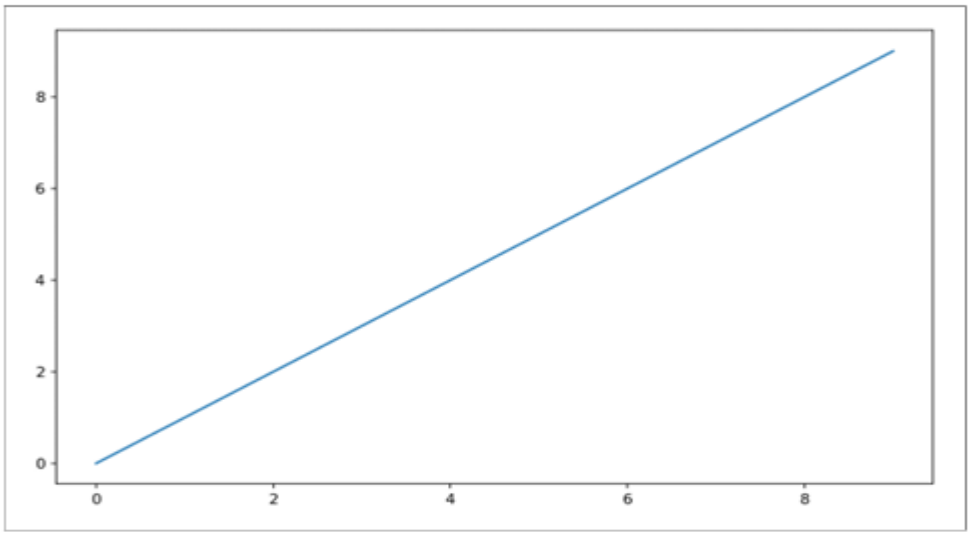



**Figure 1: Simple line plot**

This line command creates a simple line chart using default figure and subplot settings.

### A) Figure and Subplot Structure:

In matplotlib, every plot lives inside a Figure object. You can manually create a figure using:

```
fig = plt.figure()
```

At this point, the figure exists but will be empty unless you add a subplot. Subplots are added using:

```
ax1 = fig.add_subplot(2, 2, 1)
```

This means you want to divide the figure into a 2x2 grid and create (or select) the first ed similarly.

```
In [18]: ax2 = fig.add_subplot(2, 2, 2)

In [19]: ax3 = fig.add_subplot(2, 2, 3)
```

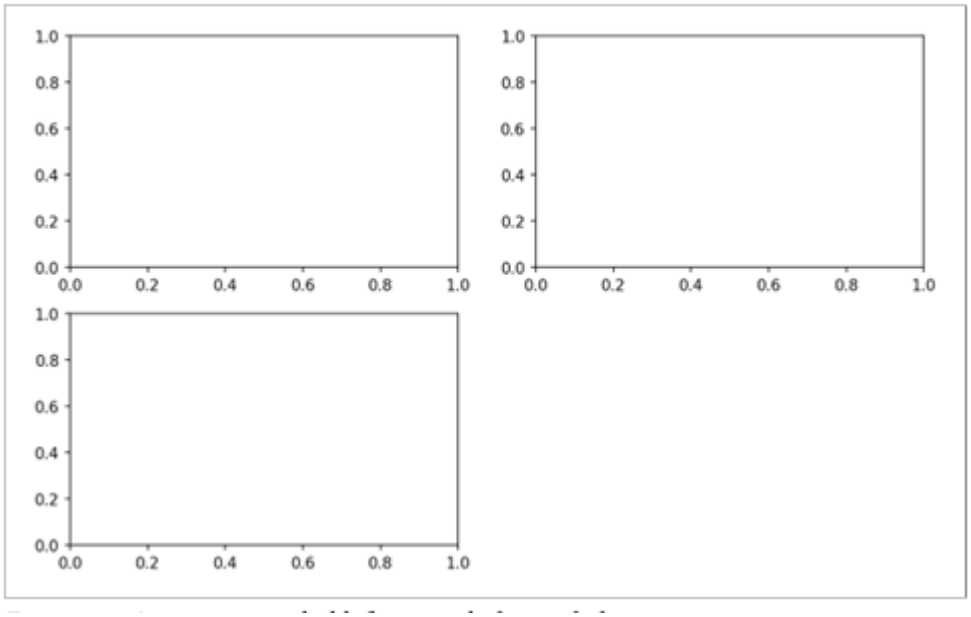



**Figure 2: An empty matplotlib figure with three subplots**

Here, run the given commands in the same cell:

fig = plt.figure()

ax1 = fig.add_subplot(2, 2, 1)

ax2 = fig.add_subplot(2, 2, 2)

ax3 = fig.add_subplot(2, 2, 3)

plt.plot([1.5, 3.5, -2, 1.6])

While executing, mat plotlib draws on the last figure and subplot used. So, add the following command, to create subplot like Figure 3.

plt.plot(np.random.randn(50).cumsum(), 'k--')

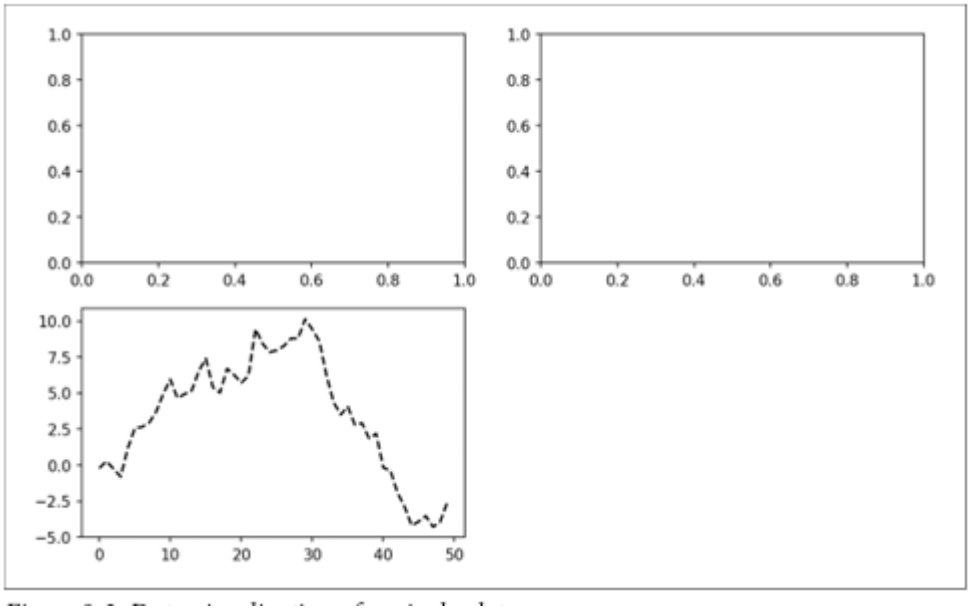


**Figure 3: Data visualization after single plot**

To style the plot, matplotlib supports format strings like `'k--'`, which plots a **black dashed line**:

```
In [21]: _ = ax1.hist(np.random.randn(100), bins=20, color='k', alpha=0.3)

In [22]: ax2.scatter(np.arange(30), np.arange(30) + 3 * np.random.randn(30))
```

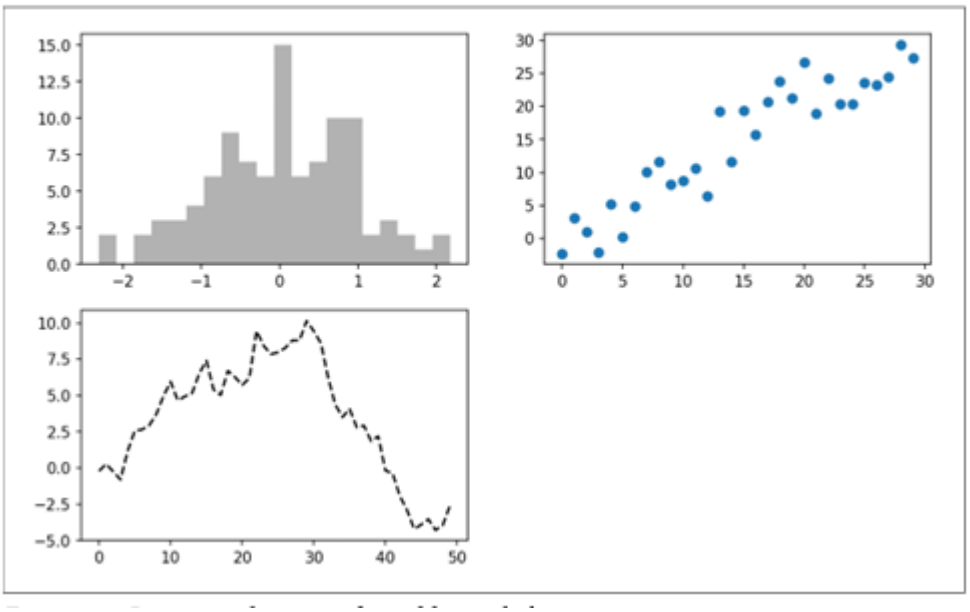


**Figure 4: Data visualization after additional plots**

The axes array can be easily indexed like a 2-D array, for example, axes[0, 1]. Like this, we can indicate the subplots with same x- or y-axis using sharex and sharey, respectively. This is

particularly useful when you compare data on the same scale; otherwise, matplotlib autoscales plot limits independently. Refer Table 1 for structure.

Table 1. pyplot.subplots options

| Argument | Description |
|---|---|
| `nrows` | Number of rows of subplots |
| `ncols` | Number of columns of subplots |
| `sharex` | All subplots should use the same x-axis ticks (adjusting the `xlim` will affect all subplots) |
| `sharey` | All subplots should use the same y-axis ticks (adjusting the `ylim` will affect all subplots) |
| `subplot_kw` | Dict of keywords passed to `add_subplot` call used to create each subplot |
| `**fig_kw` | Additional keywords to `subplots` are used when creating the figure, such as `plt.subplots(2, 2, figsize=(8, 6))` |

**B) Adjusting the spacing around subplots:**

By default, matplotlib adds spacing around and between subplots. This is scaled relative to the figure size and adjusts dynamically if you resize the plot.

To fine-tune this spacing, use the `subplots_adjust()` method. It can be called from the figure object or as a global function:

```
plt.subplots_adjust(left=..., right=..., top=..., bottom=..., wspace=..., hspace=...)
```

- `wspace` controls horizontal spacing (width between subplots)
- `hspace` controls vertical spacing (height between subplots)

**Example:**
To remove all space between subplots:

```
fig, axes = plt.subplots(2, 2, sharex=True, sharey=True)
for i in range(2):
    for j in range(2):
        axes[i, j].hist(np.random.randn(500), bins=50, color='k', alpha=0.5)
plt.subplots_adjust(wspace=0, hspace=0)
```

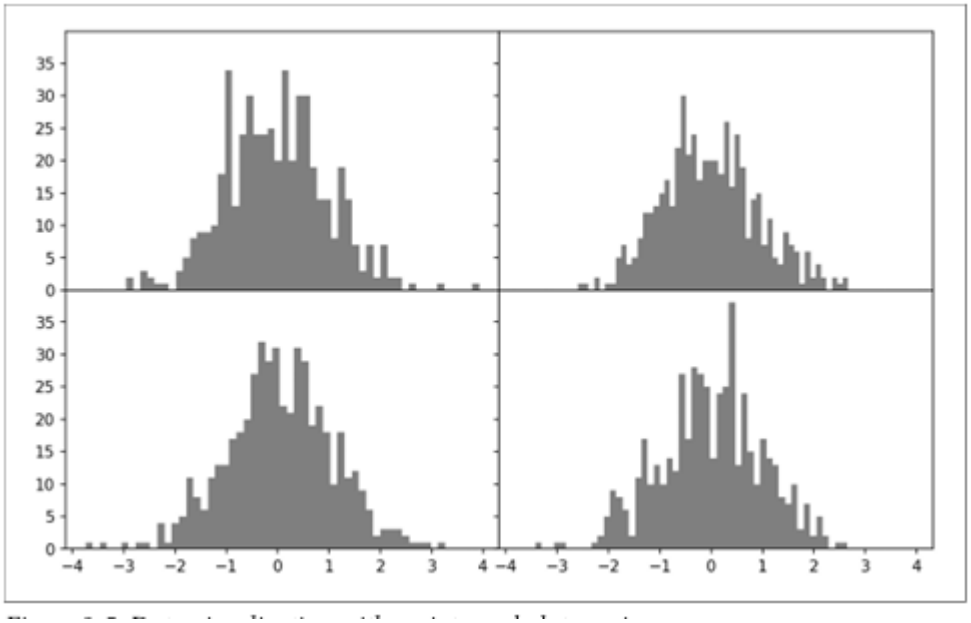



Figure 5: Data visualization with no inter-subplot spacing

### 1.2. Colors, Markers, and Line Styles

Matplotlib's core plot function accepts arrays of x and y coordinates and alternatively a string abbreviation indicating color and line style. For instance, to plot x versus y with green dashes, execute:

```
ax.plot(x, y, 'g--')
```

While combining color and line style in a single string offers convenience, it's not always ideal—especially when generating plots programmatically. In such cases, it's often better to specify styles separately rather than assembling them into one string. The same plot can be created more clearly using explicit style parameters.

```
ax.plot(x, y, linestyle='--', color='g')
```

Line charts can include markers to emphasize individual data points. Because matplotlib connects points with continuous lines, it's not always obvious where the actual data values are positioned. To make this clearer, markers can be added directly within the style string—this string typically follows

the order of color, marker type, and then line style (refer to Figure 6 for illustration).

```
In [30]: from numpy.random import randn

In [31]: plt.plot(randn(30).cumsum(), 'ko--')
```

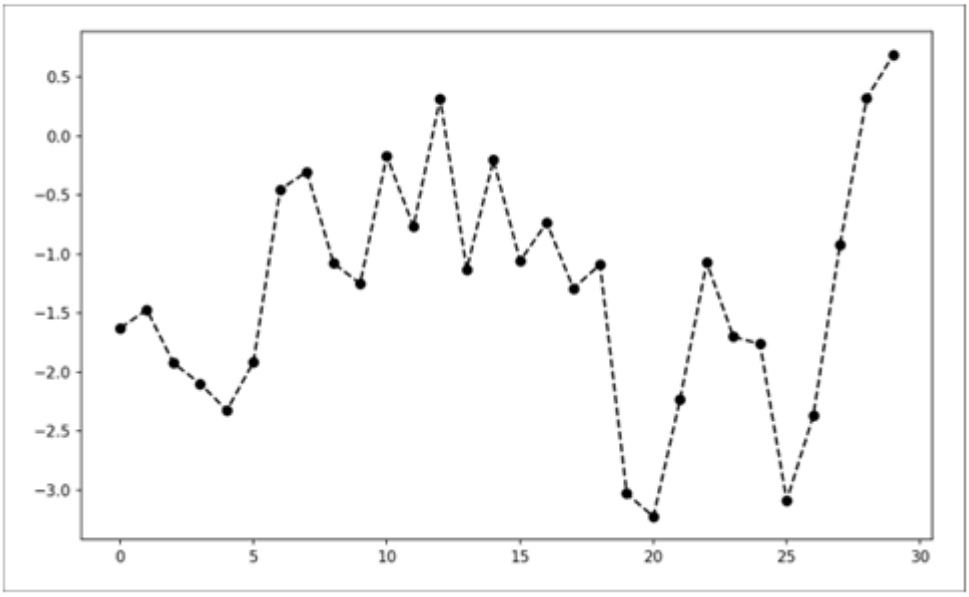



Figure 6: Line plot with markers

For line plots, subsequent points are linearly interpolated by default, which can be changed with the drawstyle option (Figure 7):

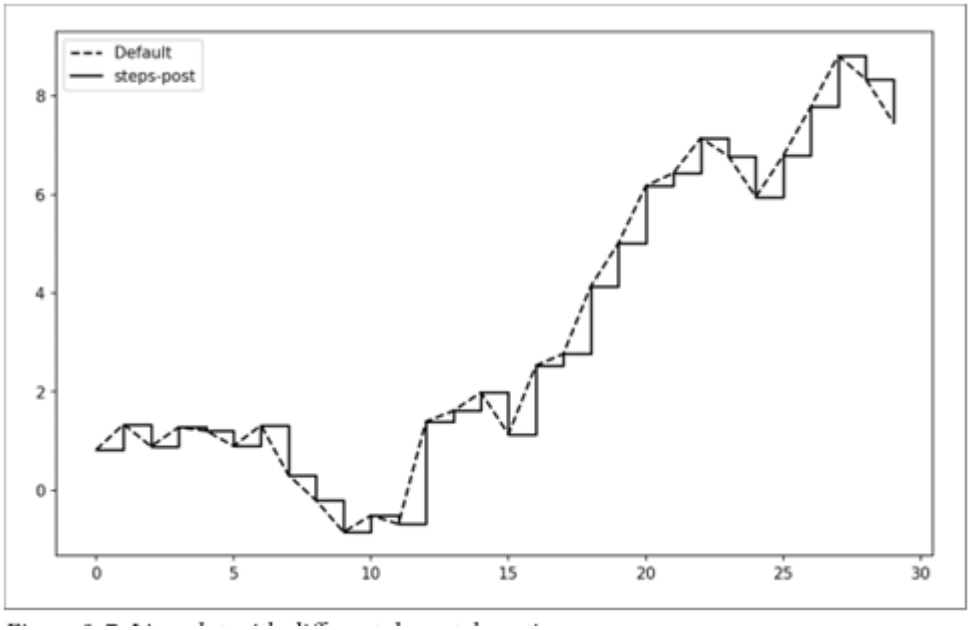



Figure 7: Line plot with different drawstyle options

### 1.3. Ticks, Labels, and Legends

When it comes to customizing plots in matplotlib, there are generally two approaches you can take: the **procedural method** using the `pyplot` module (e.g., `matplotlib.pyplot`), or the more **object-oriented method** using the core matplotlib API.

The `pyplot` interface is designed for quick, interactive plotting and includes functions like `xlim`, `xticks`, and `xticklabels`, which control aspects such as axis limits, tick positions, and tick labels.

These functions can be used in two ways:

- Without arguments: Returns the current setting (e.g., `plt.xlim()` gives the current x-axis limits).
- With arguments: Updates the setting (e.g., `plt.xlim([0, 10])` sets the x-axis range from 0 to 10).

By default, these functions apply to the most recently active `AxesSubplot` object. Internally, each of these commands has an equivalent pair of methods on the subplot object itself. For example, `xlim` corresponds to `ax.get_xlim()` and `ax.set_xlim()`.

While both methods are valid, using the object-oriented approach with subplot instances (e.g., `ax.set_xlim`) is often clearer—especially when dealing with multiple subplots. It allows for more precise control, though users can choose whichever style they find more comfortable.

A) **Setting the title, axis labels, ticks, and ticklabels**

Refer, figure 8 for customizing the axws.

```
In [37]: fig = plt.figure()

In [38]: ax = fig.add_subplot(1, 1, 1)

In [39]: ax.plot(np.random.randn(1000).cumsum())
```

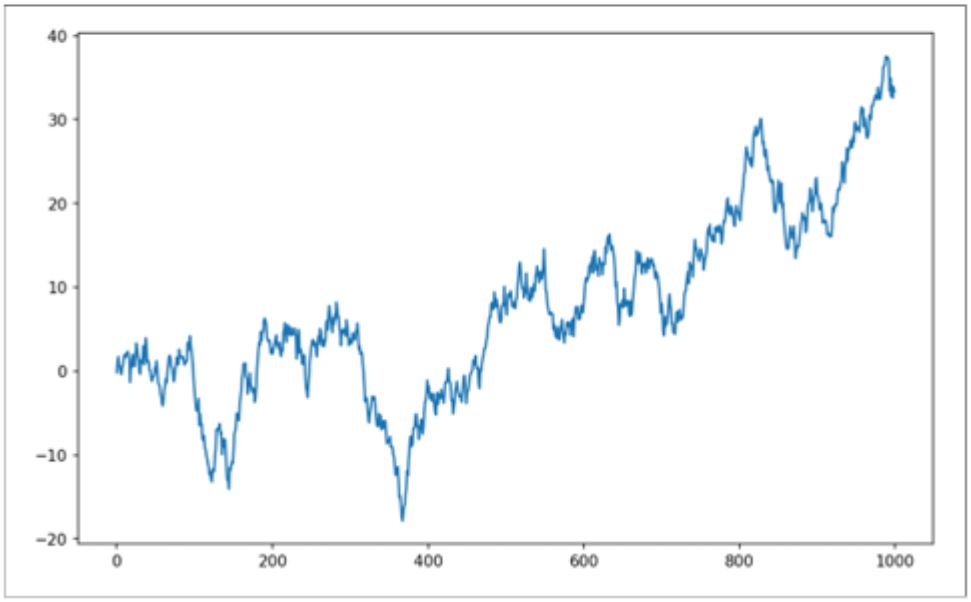



**Figure 8:** Simple plot for illustrating xticks (with label)

To change the x-axis ticks, use set_xticks and set_xticklabels.

Example,

In [40]: ticks = ax.set_xticks([0, 250, 500, 750, 1000])

In [41]: labels = ax.set_xticklabels(['one', 'two', 'three', 'four', 'five'], ….: rotation=30, fontsize='small')

The rotation option rotates the x tick labels at a 30-degree. Lastly, set_xlabel gives a name to the x-axis and set_title the subplot title (Refer figure 9)

: In [42]: ax.set_title('My first matplotlib plot')

Out[42]: In [43]: ax.set_xlabel('Stages')

In [43]: ax.set_xlabel('Stages')

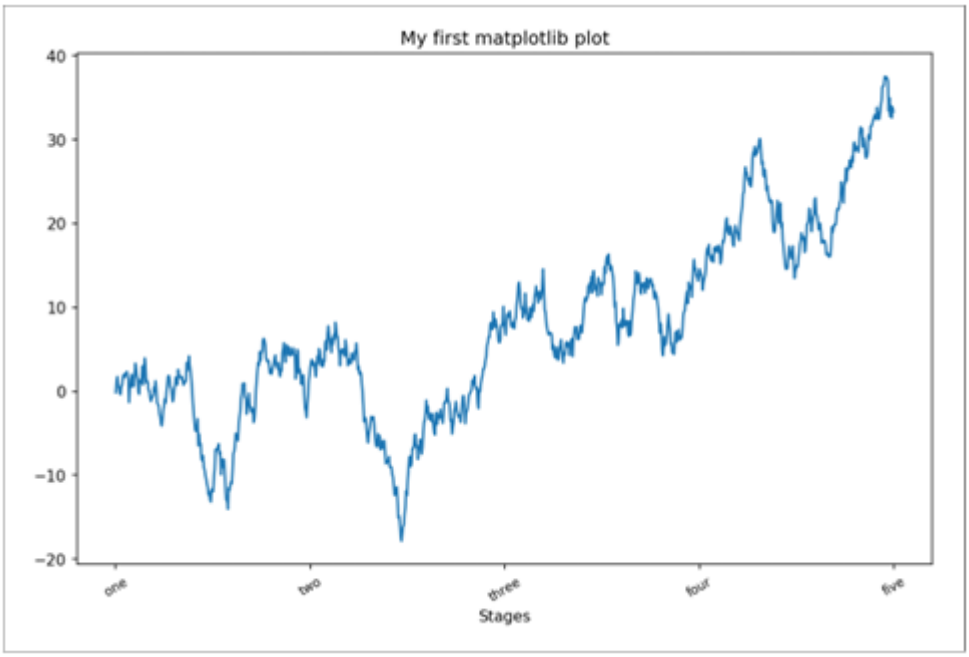



**Figure 9:** Simple plot for illustrating xticks

B) **Adding legends**

Legends are one of the critical elements to identify plot elements. There are a many ways, and the easiest is to pass the label argument when adding each piece of the plot:

```
In [44]: from numpy.random import randn

In [45]: fig = plt.figure(); ax = fig.add_subplot(1, 1, 1)

In [46]: ax.plot(randn(1000).cumsum(), 'k', label='one')
Out[46]: [<matplotlib.lines.Line2D at 0x7fb624bdf860>]

In [47]: ax.plot(randn(1000).cumsum(), 'k--', label='two')
Out[47]: [<matplotlib.lines.Line2D at 0x7fb624be90f0>]

In [48]: ax.plot(randn(1000).cumsum(), 'k.', label='three')
Out[48]: [<matplotlib.lines.Line2D at 0x7fb624be9160>]
```

**After that,** either call ax.legend() or plt.legend() to create a legend automatically (Refer figure 10)

```
In [49]: ax.legend(loc='best')
```

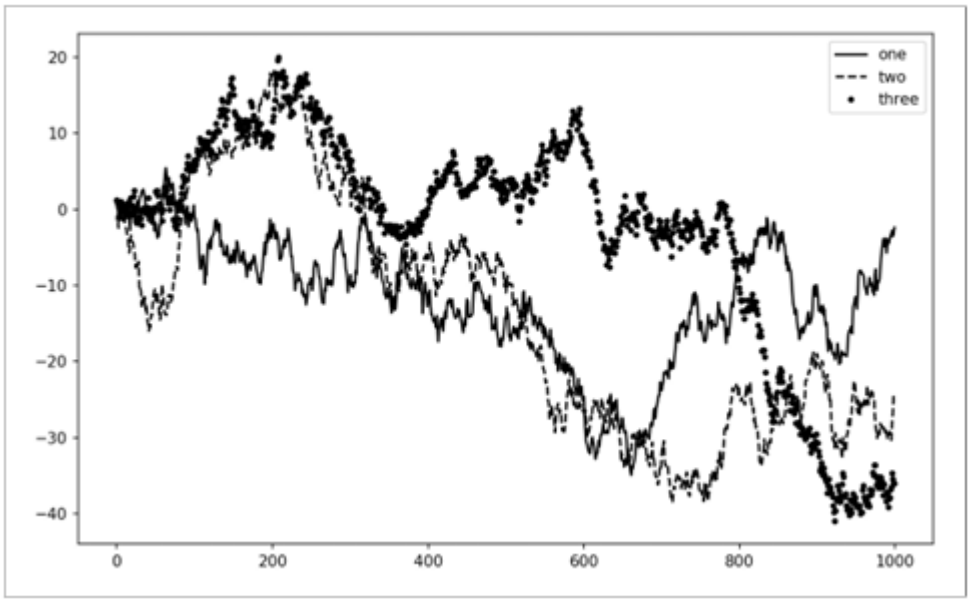



**Figure 10:** Simple plot with three lines and legend

### 1.4. Annotations and Drawing on a Subplot

To draw your plot annotations, which could consist of text, arrows, or other shapes, add annotations and text using the text, arrow, and annotate functions. 'text draws text' at specified coordinates (x, y) on the plot with optional custom styling:

```
ax.text(x, y, 'Hello world!',
        family='monospace', fontsize=10)
```

The annotations shown above add both text and arrows to the plot in a visually organized way. For instance, you could visualize the S&P 500 closing prices from 2007 onward (data available via Yahoo! Finance) and mark significant events from the 2008–2009 financial crisis directly on the graph. The entire example can be conveniently executed in a single cell within a Jupyter notebook (Refer figure 11).

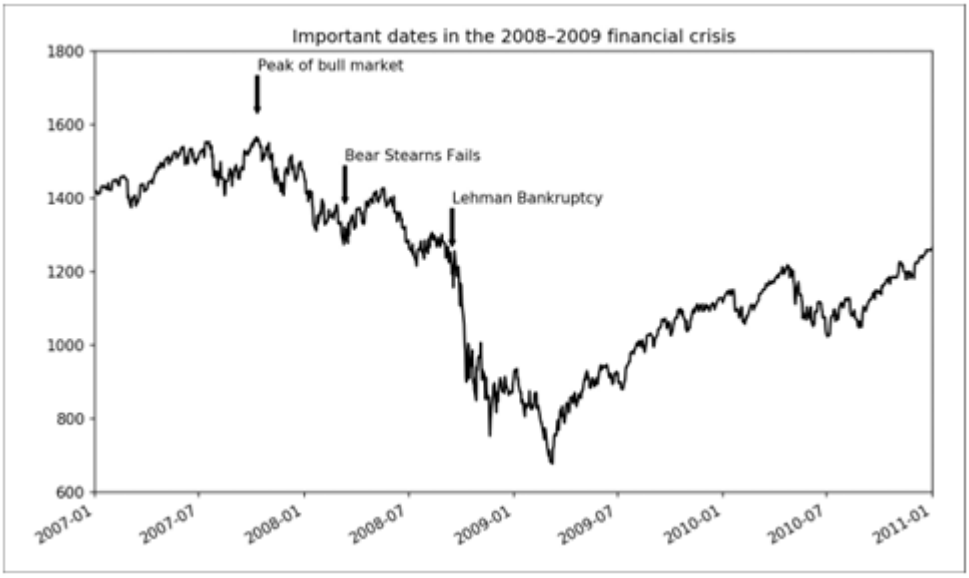



Figure 11. Important dates in the 2008–2009 financial crisis

To draw geometric shapes, matplotlib offers built-in objects for many standard forms such as Rectangle and Circle. While some are accessible through matplotlib.pyplot, the complete collection is available in the matplotlib.patches module. To include a shape in your plot, you first create the patch object (e.g., shp) and then attach it to a subplot using ax.add_patch(shp) (refer to Figure 12 for illustration).

Example:

```
fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)

rect = plt.Rectangle((0.2, 0.75), 0.4, 0.15, color='k', alpha=0.3)
circ = plt.Circle((0.7, 0.2), 0.15, color='b', alpha=0.3)
pgon = plt.Polygon([[0.15, 0.15], [0.35, 0.4], [0.2, 0.6]],
                   color='g', alpha=0.5)

ax.add_patch(rect)
ax.add_patch(circ)
ax.add_patch(pgon)
```

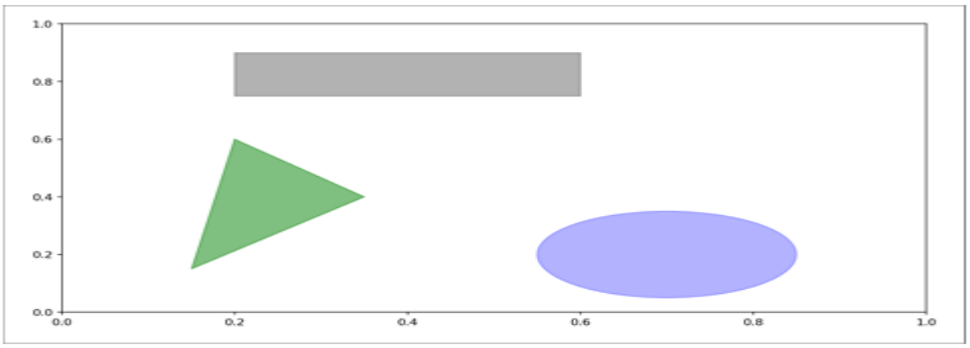



Figure 12. Data visualization composed from three different patches

### 1.5. Saving Plots to File

You can export the current figure to a file using the plt.savefig function, which performs the same action as calling the savefig method on a figure object. For instance, to save a plot in SVG format, simply use:

```
plt.savefig('figpath.svg')
```

The file type is automatically determined by the file extension you provide. So if you save it with a .pdf extension instead, a PDF file will be generated.

When preparing figures for publication, two commonly adjusted options are:

- dpi: Controls the resolution in dots per inch.
- bbox_inches='tight': Removes excess whitespace around the figure.

To save a high-resolution PNG file with minimal padding, you could write:

```
plt.savefig('figpath.png', dpi=400, bbox_inches='tight')
```

In addition to saving directly to a file, savefig can also output the plot to a file-like object such as a BytesIO stream. This is useful when you need to process the image in memory:

```
from io import BytesIO
buffer = BytesIO()
plt.savefig(buffer)
plot_data = buffer.getvalue()
```

For more customization options, refer to the full list in Table 2.

Table 2. Figure.savefig options

| Argument | Description |
|---|---|
| `fname` | String containing a filepath or a Python file-like object. The figure format is inferred from the file extension (e.g., `.pdf` for PDF or `.png` for PNG) |
| `dpi` | The figure resolution in dots per inch; defaults to 100 out of the box but can be configured |
| `facecolor`, `edgecolor` | The color of the figure background outside of the subplots; `'w'` (white), by default |
| `format` | The explicit file format to use (`'png'`, `'pdf'`, `'svg'`, `'ps'`, `'eps'`, ...) |
| `bbox_inches` | The portion of the figure to save; if `'tight'` is passed, will attempt to trim the empty space around the figure |

### 1.6. matplotlib Configuration

By default, **matplotlib** is set up with color palettes and layout options suited for creating figures intended for publication. However, nearly every default setting—such as figure dimensions, spacing between subplots, font size, color schemes, grid appearance, and more—can be fully customized.

One way to adjust these settings programmatically is by using the `rc()` function. For example, to globally set the default figure size to **10 by 10**, you would write:

```
plt.rc('figure', figsize=(10, 10))
```

The first argument passed to `rc()` specifies the component to customize (like `'figure'`, `'axes'`, `'xtick'`, `'ytick'`, `'grid'`, or `'legend'`). Additional settings can be supplied as keyword arguments.

Another convenient approach is to define settings in a dictionary and apply them with unpacking. For example:

```
font_options = {'family' : 'monospace',
                'weight' : 'bold',
                'size'   : 'small'}
plt.rc('font', **font_options)
```

For broader and more persistent customization, matplotlib provides a config file named **`matplotlibrc`**, located in the `matplotlib/mpl-data` directory. You can also create your own version of this file (named `.matplotlibrc`) and save it in your home directory. When matplotlib starts, it will automatically apply these custom settings.

As you'll learn later, the seaborn library builds on matplotlib and includes several pre-defined themes and styles, all managed through this same configuration system.

## 2. Plotting with pandas and seaborn

matplotlib operates at a relatively low level, requiring users to build a plot by combining its fundamental elements—such as the data visualization type (e.g., line, bar, scatter, box, contour), along with the title, axis labels, legends, and other annotations.

In contrast, pandas makes visualization easier, especially when working with DataFrames or Series that include multiple data columns and labeled rows and columns. It offers built-in plotting functions that streamline the process.

Additionally, the seaborn library—developed by Michael Waskom—is a high-level interface built on top of matplotlib. It simplifies the creation of commonly used statistical plots, making it easier to produce attractive and informative graphics.

### 2.1. Line Plots

Both **Series** and **DataFrame** objects in **pandas** include a `plot` attribute that allows users to create basic visualizations. When you call the `plot()` method without specifying a plot type, it defaults to generating a **line plot** (as shown in Figure 13).

```
In [60]: s = pd.Series(np.random.randn(10).cumsum(), index=np.arange(0, 100, 10))

In [61]: s.plot()
```

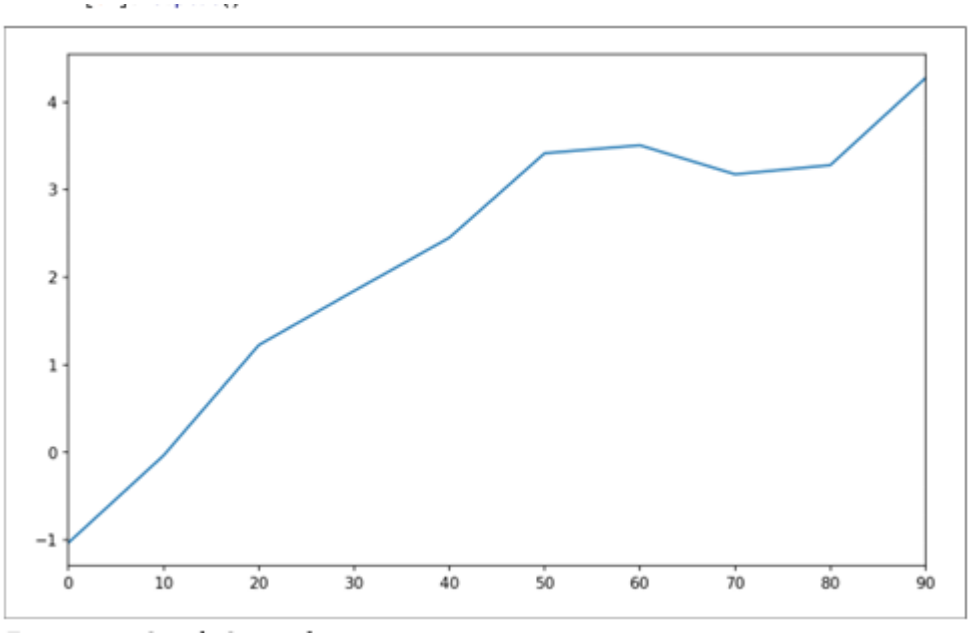



Figure 13. Simple Series plot

By default, the Series index is used as the x-axis values in the plot. If you prefer not to use the index, you can disable it by setting `use_index=False`. Axis tick marks and range limits can be customized using parameters such as `xticks` and `xlim` for the x-axis, and `yticks` and `ylim` for the y-axis. Table 3 provides a complete list of available plotting options. While some of these options are explained further in this section, you are encouraged to explore the rest independently.

Additionally, most plotting functions in pandas accept an optional `ax` argument, which allows you to pass a matplotlib subplot object**.** This is especially helpful when organizing multiple plots within a grid layout.

When using `plot()` on a DataFrame**,** each column is plotted as a separate line within the same subplot, and a legend is automatically generated (as demonstrated in Figure 14).

```
In [62]: df = pd.DataFrame(np.random.randn(10, 4).cumsum(0),
   ....:                   columns=['A', 'B', 'C', 'D'],
   ....:                   index=np.arange(0, 100, 10))

In [63]: df.plot()
```

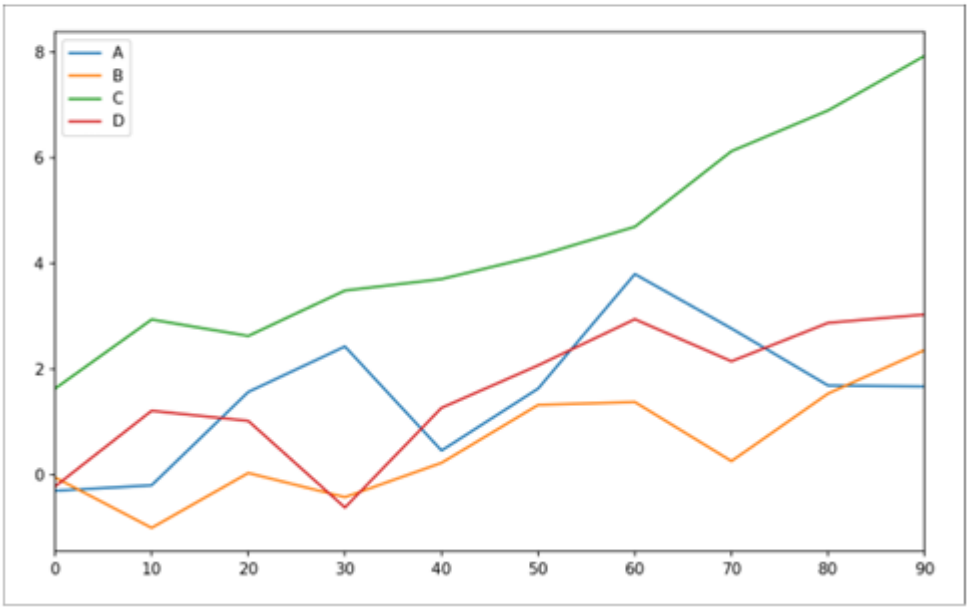



Figure 14. Simple DataFrame plot

Table 3. Series.plot method arguments

| Argument | Description |
|---|---|
| `label` | Label for plot legend |
| `ax` | matplotlib subplot object to plot on; if nothing passed, uses active matplotlib subplot |
| `style` | Style string, like `'ko--'`, to be passed to matplotlib |
| `alpha` | The plot fill opacity (from 0 to 1) |
| `kind` | Can be `'area'`, `'bar'`, `'barh'`, `'density'`, `'hist'`, `'kde'`, `'line'`, `'pie'` |
| `logy` | Use logarithmic scaling on the y-axis |
| `use_index` | Use the object index for tick labels |
| `rot` | Rotation of tick labels (0 through 360) |
| `xticks` | Values to use for x-axis ticks |
| `yticks` | Values to use for y-axis ticks |
| `xlim` | x-axis limits (e.g., `[0, 10]`) |
| `ylim` | y-axis limits |
| `grid` | Display axis grid (on by default) |

DataFrame allows some flexibility to handle the columns; for example, whether to plot all columns on the same subplot or to create separate subplots (refer to table 4 for more details).

Table 4. DataFrame-specific plot arguments

| Argument | Description |
|---|---|
| subplots | Plot each DataFrame column in a separate subplot |
| sharex | If subplots=True, share the same x-axis, linking ticks and limits |
| sharey | If subplots=True, share the same y-axis |
| figsize | Size of figure to create as tuple |
| title | Plot title as string |
| legend | Add a subplot legend (True by default) |
| sort_columns | Plot columns in alphabetical order; by default uses existing column order |

## 2.2. Bar Plots

To create vertical and horizontal bar plots, use plot.bar() and plot.barh() respec tively. Here, the Series or DataFrame index will use as the x (bar) or y (barh) ticks (Figure 15):

```
In [64]: fig, axes = plt.subplots(2, 1)

In [65]: data = pd.Series(np.random.rand(16), index=list('abcdefghijklmnop'))

In [66]: data.plot.bar(ax=axes[0], color='k', alpha=0.7)
Out[66]: <matplotlib.axes._subplots.AxesSubplot at 0x7fb62493d470>

In [67]: data.plot.barh(ax=axes[1], color='k', alpha=0.7)
```

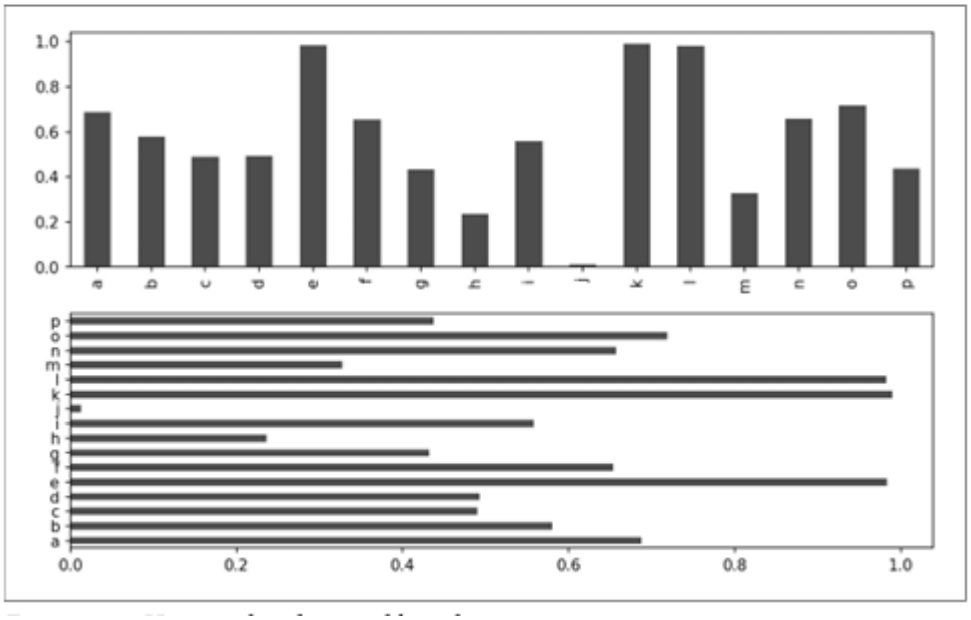



Figure 15. Horizonal and vertical bar plot

The options such as color='k' and alpha=0.7, gives black color to the plots and use partial transparency for filling.

When creating a bar plot from a DataFrame, the values from each row are displayed as grouped bars, positioned side by side to represent each data point (Figure 16).

```
In [69]: df = pd.DataFrame(np.random.rand(6, 4),
   ....:                   index=['one', 'two', 'three', 'four', 'five', 'six'],
   ....:                   columns=pd.Index(['A', 'B', 'C', 'D'], name='Genus'))

In [70]: df
Out[70]:
Genus         A         B         C         D
one    0.370670  0.602792  0.229159  0.486744
two    0.420082  0.571653  0.049024  0.880592
three  0.814568  0.277160  0.880316  0.431326
four   0.374020  0.899420  0.460304  0.100843
five   0.433270  0.125107  0.494675  0.961825
six    0.601648  0.478576  0.205690  0.560547

In [71]: df.plot.bar()
```

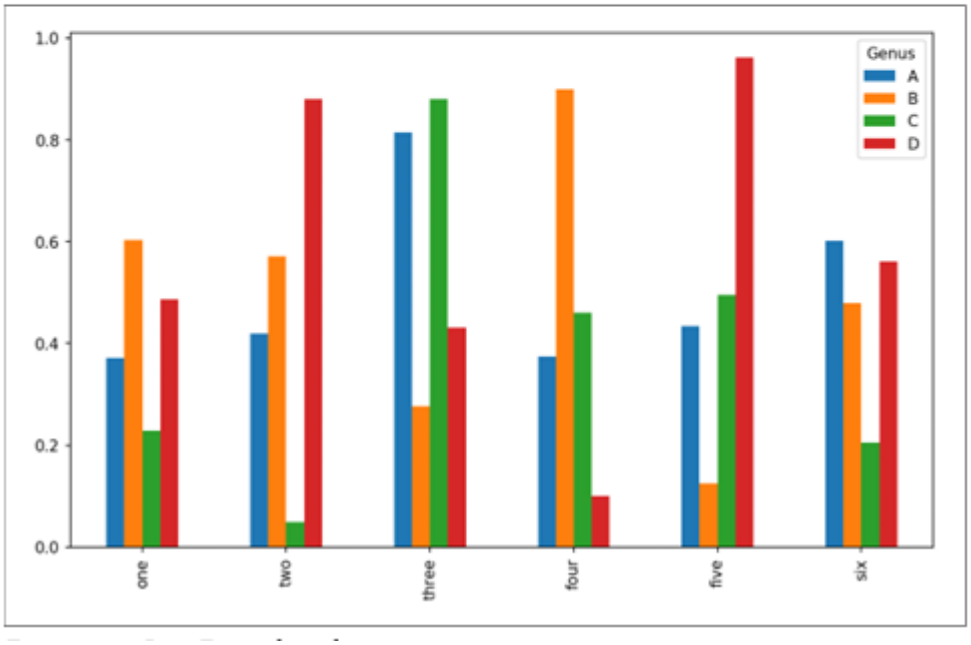



Figure 16. DataFrame bar plot

Here, “Genus” is the title of the legend

To generate stacked bar plots from a DataFrame use stacked=True (Figure 17)

```
In [73]: df.plot.barh(stacked=True, alpha=0.5)
```

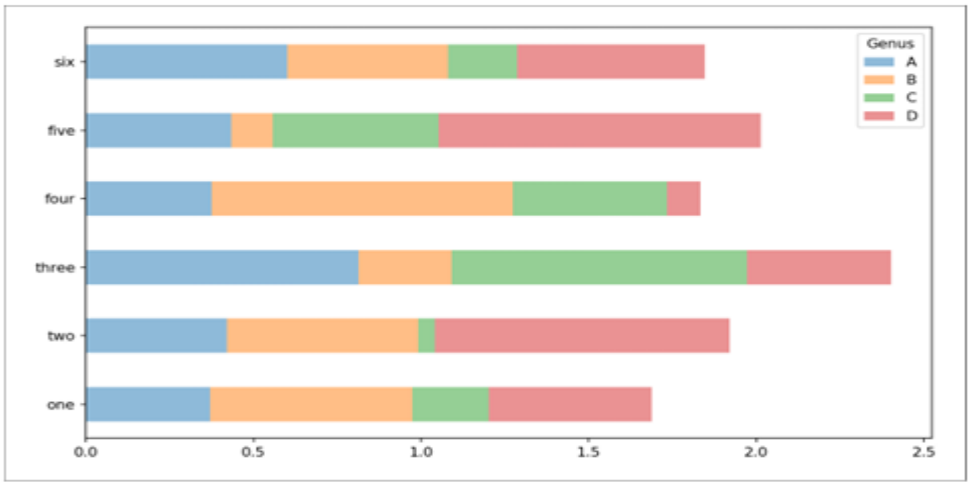


Figure 17. DataFrame stacked bar plot

To make a stacked bar plot showing the percentage of data points (for example, each party size on each day)

```
In [75]: tips = pd.read_csv('examples/tips.csv')

In [76]: party_counts = pd.crosstab(tips['day'], tips['size'])

In [77]: party_counts
Out[77]:
size  1   2   3   4  5  6
day
Fri   1  16   1   1  0  0
Sat   2  53  18  13  1  0
Sun   0  39  15  18  3  1
Thur  1  48   4   5  1  3


# Not many 1- and 6-person parties
In [78]: party_counts = party_counts.loc[:, 2:5]
```

Then, normalize (Figure: 18)

```
# Normalize to sum to 1
In [79]: party_pcts = party_counts.div(party_counts.sum(1), axis=0)

In [80]: party_pcts
Out[80]:
size         2         3         4         5
day
Fri   0.888889  0.055556  0.055556  0.000000
Sat   0.623529  0.211765  0.152941  0.011765
Sun   0.520000  0.200000  0.240000  0.040000
Thur  0.827586  0.068966  0.086207  0.017241

In [81]: party_pcts.plot.bar()
```

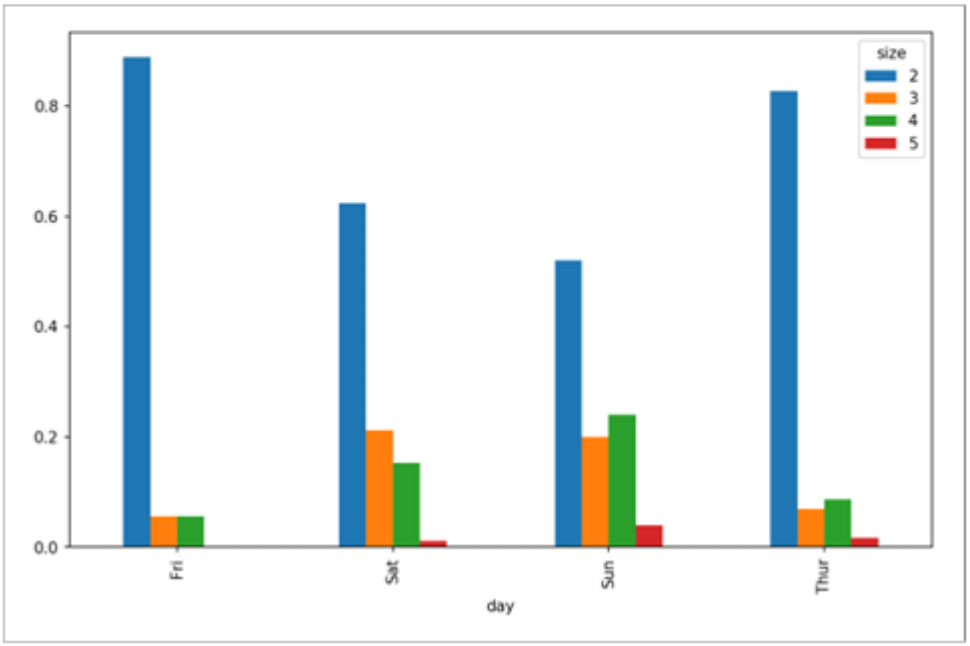


Figure 18. Fraction of parties by size on each day

For aggregation or summarization before making a plot, using the seaborn package. For example, tipping percentage by day (Figure: 19)

```
In [83]: import seaborn as sns

In [84]: tips['tip_pct'] = tips['tip'] / (tips['total_bill'] - tips['tip'])

In [85]: tips.head()
Out[85]:
   total_bill   tip smoker  day    time  size   tip_pct
0       16.99  1.01     No  Sun  Dinner     2  0.063204
1       10.34  1.66     No  Sun  Dinner     3  0.191244
2       21.01  3.50     No  Sun  Dinner     3  0.199886
3       23.68  3.31     No  Sun  Dinner     2  0.162494
4       24.59  3.61     No  Sun  Dinner     4  0.172069

In [86]: sns.barplot(x='tip_pct', y='day', data=tips, orient='h')
```

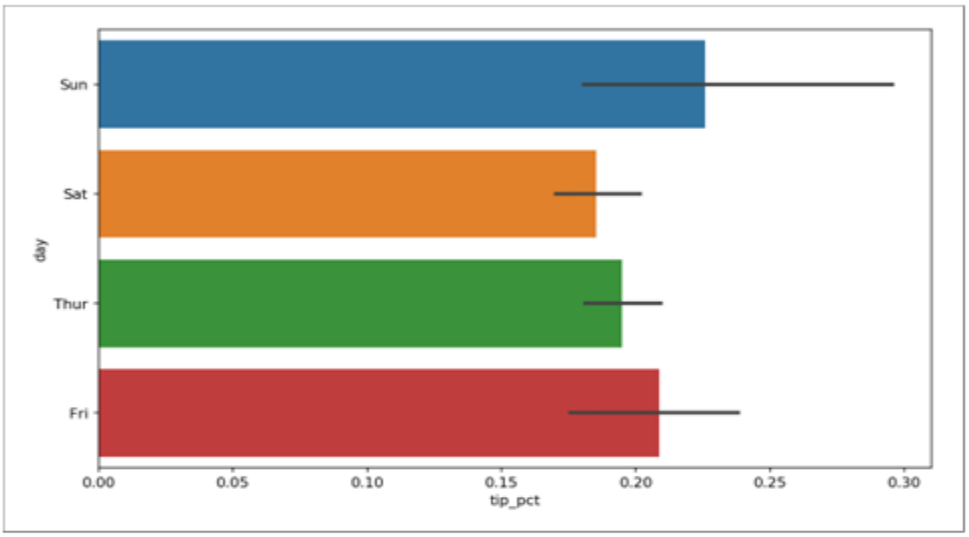



Figure 19. Tipping percentage by day with error bars

To split by an additional categorical value, seaborn.barplot use the hue option (Figure 20).

```
In [88]: sns.barplot(x='tip_pct', y='day', hue='time', data=tips, orient='h')
```

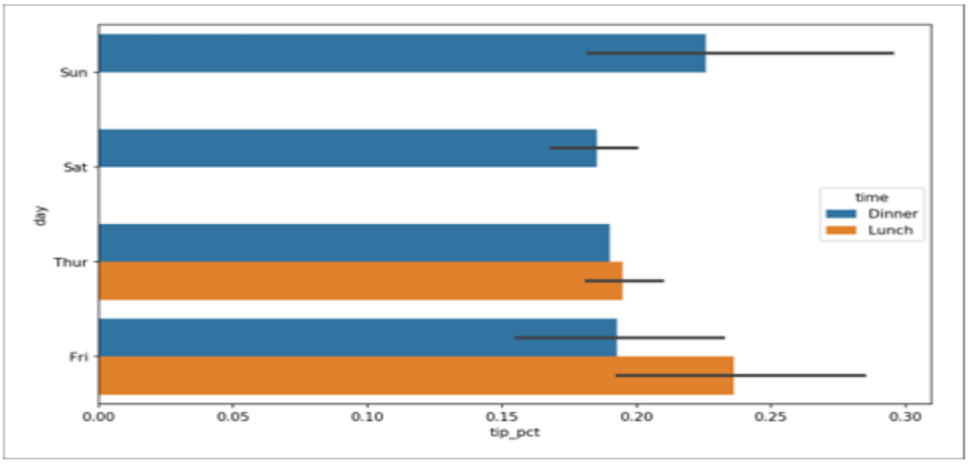



Figure 20. Tipping percentage by day and time

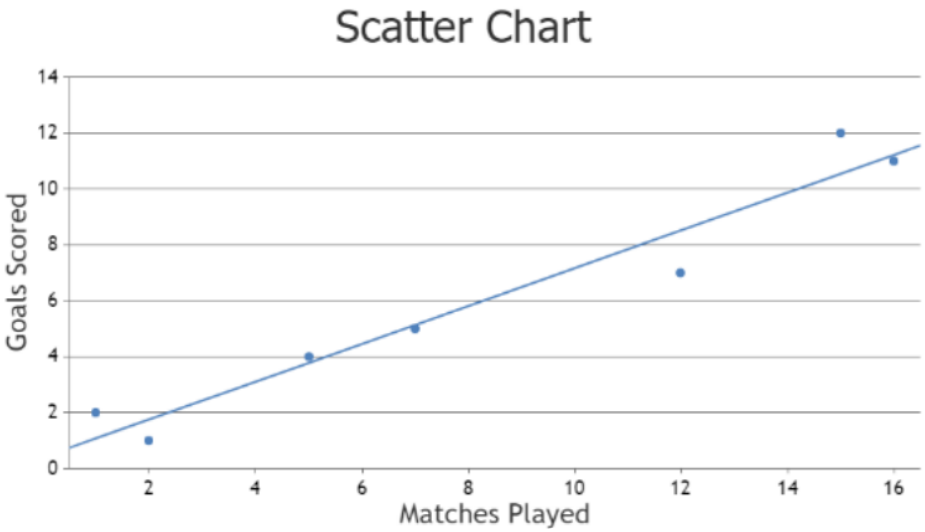
## Scatter plot :

One of the most significant data visualization methods is scatter plot and it is regarded to be one of the Seven Basic Tools of Quality. The relationship of two variables is plotted in the chart called a scatter plot on two dimensional graph commonly referred to as Cartesian Plane on mathematical basis.

It is normally employed to draw the association between one independent variable and one dependent variable with an independent variable plotted on x- axis and a dependent variable plotted on y-axis so that you can determine the influence that the independent variable has on the dependent variable. These are called Scatter Plot Graph, or Scatter Diagram.

Scatter Plot has other names and some of them include; scatter chart, scattergram, scatter plot, and XY graph. Scatter plot is the graphical representation of a data pair where each element of the pair is assigned an axis that normally is the independent axis, called the x-axis and the other axis, the dependent axis, called the y-axis.

Such type of distribution facilitates the easy determination of the type of relationship, the data pair in plotted format is assuming. Thus the Scatter Plot comes in handy when we have to determine the relationship between any two sets of data or in cases where we are suspecting that there might be some relationship between the two variables and the relationship might be the gist of a problem being encountered somewhere.




We need to observe the steps below in order to create a scatter plot.

Identify the independent and dependent variables (step 1)

Step 2: On x-axis put the independent variable

Step 3: Leave the y-axis blank and plots the dependent variable

Step 4 The significant connection between the provided variables should be pulled out.

Forms of Scatter Plot

The Scatter Plot on the basis of correlation of two variables will fall under the types as following.

Scatter Plot Of Positive Correlation

Scatter Plot Of Negative Correlation

Scatter Plot of Null Correlation

Scatter Plot of Positive Correlation

In such kind of scatter-plot values on Y axis rises as one moves along from left to right. Technically speaking, when two variables are directly proportional to each other then there will be positive correlation as portrayed in the scatter plot. The familiarity is further discussed in the form of Perfect Positive, High Positive and Low Positive.

Scatter Plot Of Negative Correlation

In such scatter-plot, the y measure declines as we go left to right. Valuation of one variable against the other is reducing, in layman terms. There are further forms of positive correlation that are referred to as Perfect Negative, High Negative and Low Negative.

Scatter Plot of Null Correlation

Values in such a category of scatter-plot are distributed everywhere in the graph. Generally such a type of graph indicates that there exists no correlation between the two variables drawn in the Scatter Plot.

What is an Analysis of Scatter Plot?

The use of scatter plot analysis entails analysis of how the points on the graph are distributed and arriving to the general picture in order to get insights into the association between the variables. Scatter Plot helps to visualise correlated variables of two variables; however, in life, we are not that lucky to obtain only a correlation between variables. Such are the cases in real life cases, and more than two variables can be the part of correlation.

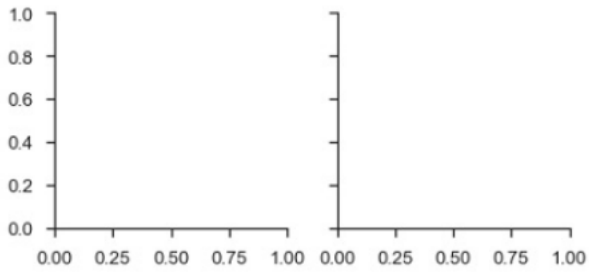
## **Facet grids and Categorical data**:

FacetGrid object requires a dataframe and the labels of the variables which will appear in the row, column or hue dimensions of the grid.

The variables must be categorical and the data at every level of the variable is to be used as a facet along that dimension.In the given example we have simply created the facetgrid object which does not draw on them.

Forming visual data of this grid is mainly done using the FacetGrid.map() approach. Let us turn to the tip distributions on each of these subsets, a histogram can be used.

```
import pandas as pd
import seaborn as sb
from matplotlib import pyplot as plt
df = sb.load_dataset('tips')
g = sb.FacetGrid(df, col = "time")
plt.show()
```




## Other Python visualization tools

In data science, distribution of your data is very important. Histograms are useful in visualizing frequency distributions as are box plots (box-and-whisker). Violin plots deliver one step further the box plots in combination with the kernel density estimate (KDE), providing a more detailed outlook into distribution shape. It is also useful that KDE plots can easily be used to display smoothly calculated distributions, particularly when we want to compare several groups of data.

In categorical data, strip plot and swarm plot give the information about data distribution inside categories. Strip plots provide not-overlapping points by making it rougher (jitter) whilst swarm plot arrange the data in order to avoid overlapping at all, these are the best when one has small-to-medium provided data to work with. The stacked area chart is applied when comparing over time the composition of several categories. When data is hierarchical, then the proportion is visualized in tree maps and sunburst charts, and it is useful in showing categories and subcategories.

When dealing with geographical data, one idea is that of color-coded regions (such as countries or states) these processes work out well via the use of choropleth maps. Point maps can be used to see a specific geographical location on a latitude-longitude scale in maps and heatmaps are maps that show the intensity of data, like the number of people in some locations. These involve libraries such as folium, geopandas, plotly to make the tasks easier.

When working with analyzed relationships or interactions (e.g. social networks or web traffic), network graphs can be very helpful. You can render interconnectivity between entities (node) and connections (edge) using networkx or pyvis in an interactive way. On the same note, correlation tables that are sometimes represented in the form of heat maps can make you rapidly understand the quality of the association between numerical variables. Hierarchical clustering utilizes dendrograms, which are used to indicate the formation of data clusters depending on similarity.